

Aula Técnica — Smart Contracts ContractEase

Para quem é este material: Lucas, fundador do ContractEase. **Objetivo:** Entender, de ponta a ponta, o que foi construído na pasta `soroban-contracts/` e como integrar com o frontend. Ao final, você consegue ler o código Rust, adicionar novos contratos, fazer deploy e debugar problemas em produção.

Sumário

- [1. O que é Soroban e por que escolhemos](#)
- [2. Arquitetura geral do sistema](#)
- [3. Anatomia de um smart contract Rust](#)
- [4. Os 6 contratos linha a linha](#)
 - [4.1 Rent \(Aluguel\)](#)
 - [4.2 Ecommerce \(Venda Online\)](#)
 - [4.3 Freelancer](#)
 - [4.4 Legal Fees \(Honorários\)](#)
 - [4.5 Construction \(Empreitada\)](#)
 - [4.6 Real Estate Token \(Tokenização Imobiliária\)](#)
- [5. Como funciona o deploy ponta a ponta](#)
- [6. Frontend: chamando os contratos](#)
- [7. Testando localmente](#)
- [8. Como criar um novo contrato \(passo a passo\)](#)
- [9. Glossário](#)
- [10. Troubleshooting](#)

1. O que é Soroban e por que escolhemos

Soroban é a plataforma de smart contracts da Stellar. Diferente do Ethereum (Solidity / EVM), ela tem 3 características que importam:

Característica	Stellar/Soroban	Ethereum/Solidity
Linguagem	Rust → WASM	Solidity → EVM bytecode
Custo de tx	<\$0.001 (fração de centavo)	~\$1-30 (gas volátil)
Tempo de confirmação	3-5 segundos	15-30 segundos
Storage	Caro mas previsível	Muito caro
Patrocínio de fees	Nativo (sponsor)	Não nativo

Para o ContractEase isso significa: o usuário **não precisa ter XLM** para usar o sistema — o sponsor patrocina. E ele paga centavos por qualquer ação (assinar, pagar parcela, abrir disputa).

Mental model em uma frase

Um smart contract é um **objeto stateful** publicado na blockchain. Tem dados (storage), funções públicas (`pub fn`), e regras que garantem que só quem está autorizado consegue mexer.

2. Arquitetura geral do sistema



```
real-estate-vault/ ← Vault de cotação imobiliária

scripts/build-all.sh → gera ./dist/*.wasm
scripts/upload-wasms.sh → sobe para Supabase Storage
```

O que entra no bucket vs. o que vai pra blockchain

- **Bucket Supabase** `contracts-wasm/` — guarda o `.wasm` (bytecode Rust compilado). É o "template" que será deployado N vezes.
- **Blockchain Stellar** — cada deploy cria uma **instância** do contrato com endereço próprio (`C...`) e storage independente.

Pense: o WASM no bucket é a **classe Java**. Cada deploy é um **objeto** instanciado dessa classe.

3. Anatomia de um smart contract Rust

Vamos dissecar o esqueleto. Abra `soroban-contracts/rent/src/lib.rs` em paralelo.

3.1 Imports e flags do compilador

```
#![no_std]

use contractease_common::{ ... };
use soroban_sdk::{ contract, contractimpl, contracttype, ... };
```

- `#![no_std]` — Soroban roda em WASM com runtime mínimo. Sem `std::Vec`, sem `std::HashMap`. Usamos `soroban_sdk::Vec`, `soroban_sdk::Map`.
- `contractease_common` — nossa lib compartilhada (erros, helpers).

3.2 Storage keys

```
const DATA: Symbol = symbol_short!("DATA");
const MONTH: Symbol = symbol_short!("MONTH");
```

Cada chave no storage on-chain é um `Symbol` (string curta até 9 chars). Convenção: keys em CAIXA ALTA.

3.3 Tipos de erro

```
#[contracterror]
#[derive(Copy, Clone, Debug, PartialEq, Eq, PartialOrd, Ord)]
#[repr(u32)]
pub enum RentError {
    DepositAlreadyPaid = 100,
    RentAlreadyPaidThisMonth = 101,
    ...
}
```

Cada erro tem um número estável. Quando o frontend recebe `Error(Contract, #100)`, ele mapeia para "Caução já foi paga".

Convenção: erros comuns (1-99) ficam em `common::CommonError`. Erros específicos do contrato começam em 100.

3.4 Tipos de dado

```
#[contracttype]
#[derive(Clone, Debug, PartialEq, Eq)]
pub enum RentState {
    AwaitingDeposit,
    Active,
    Overdue,
    ...
}

#[contracttype]
#[derive(Clone)]
pub struct RentalAgreement {
    pub landlord: Address,
    pub tenant: Address,
    pub asset: Address,
    pub monthly_rent: i128,
    ...
    pub state: RentState,
}
```

`#[contracttype]` serializa o tipo automaticamente para XDR (o formato binário do Stellar). Tudo que vai pro storage precisa ter esse atributo.

3.5 Função `init` (construtor)

```
pub fn init(env: Env, params: RentInitParams) {
    if env.storage().instance().has(&DATA) {
        panic_with_error(&env, CommonError::AlreadyInitialized);
    }
    // ... validações ...
    let agreement = RentalAgreement { ... };
    env.storage().instance().set(&DATA, &agreement);
}
```

- Roda **uma única vez**, logo após o deploy.
- `env.storage().instance()` — storage permanente do contrato (TTL estendido automaticamente quando você o lê).
- O guard `if has(&DATA)` previne re-init.

3.6 Função de transição de estado

```
pub fn pay_deposit(env: Env) {
    let mut d: RentalAgreement = load(&env);
```

```

require_state(&env, d.state == RentState::AwaitingDeposit);
d.tenant.require_auth();

token_transfer(&env, &d.asset, &d.tenant, &env.current_contract_address(), d.deposit);

d.state = RentState::Active;
save(&env, &d);

env.events().publish((symbol_short!("deposit"),), d.deposit);
}

```

Padrão de toda função pública:

1. **Load** — recupera o estado do storage.
2. **Validate state** — só pode ser chamada se o contrato estiver no estado certo.
3. **require_auth()** — força a Stellar a validar que essa wallet assinou a transação.
4. **Lógica** — transfere tokens, atualiza dados, valida invariantes.
5. **Save** — grava o novo estado.
6. **Emit event** — para o frontend escutar.

3.7 Funções read-only (sem custo)

```

pub fn get_state(env: Env) -> RentState {
    load(&env).state
}

```

Não modificam estado. Podem ser chamadas via `simulateTransaction` (gratuito, sem precisar de wallet).

3.8 Testes

```

#[cfg(test)]
mod test;

```

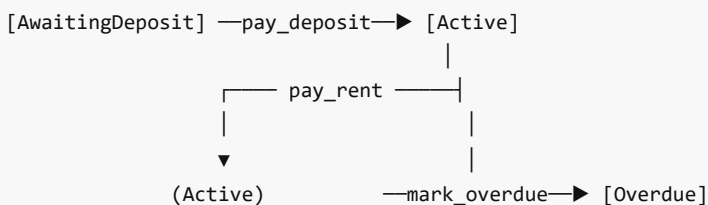
Os testes ficam em `src/test.rs`. Eles rodam **fora** da blockchain usando o `soroban_sdk::testutils::Env::default()`, que simula o ambiente.

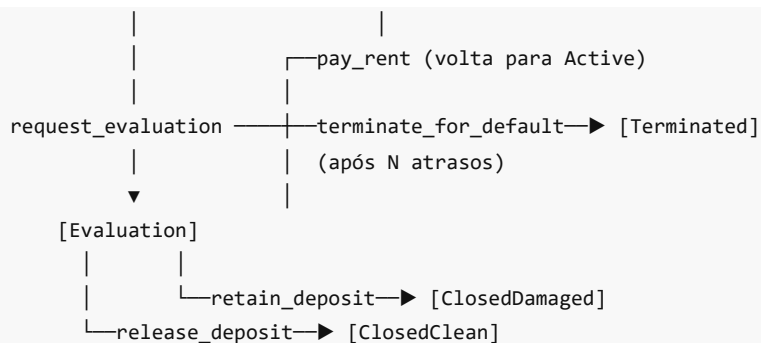
4. Os 6 contratos linha a linha

4.1 Rent (Aluguel Residencial)

Arquivo: `soroban-contracts/rent/src/lib.rs`

Estado e transições





Mecânica de mora

```

let days_late = (now - due_ts) / SECONDS_PER_DAY;
late_fee = d.monthly_rent
    .checked_mul(d.late_fee_bps as i128)
    .and_then(|x| x.checked_mul(days_late as i128))
    .and_then(|x| x.checked_div(BPS_DENOMINATOR.checked_mul(30).unwrap()))
    .unwrap_or(0);
  
```

Tradução: multa = aluguel × bps × dias_atraso / (10000 × 30) . Em bps (basis points), 200 = 2%. Então 2% ao mês, proporcional ao dia.

Por que i128 para valores?

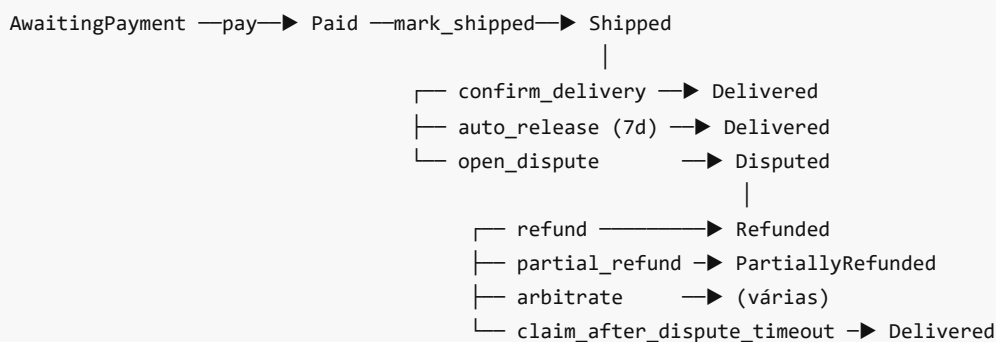
i128 aguenta 1.7×10^{38} . Para um sistema de aluguel, isso é overkill, mas Soroban padroniza tudo em i128 para não ter overflow em casos como tokenização imobiliária (cotas × preço × distribuições mensais).

Lembre: tudo em **stroops** (10^7). R\$ 2.500,00 = 25_000_000_000 stroops.

4.2 Ecommerce (Venda Online)

Arquivo: soroban-contracts/ecommerce/src/lib.rs

Estados



Anti-troll: timeout de disputa

Comprador abre disputa e some? Vendedor pode reivindicar após `dispute_resolution_days` :

```
if now < s.dispute_opened_ts + s.dispute_resolution_secs {
    panic_with_error_ecom(&env, EcomError::DisputeTimeoutNotReached);
}
```

Árbitro opcional

`arbiter: Option<Address>` . Se passado no init, ele pode decidir `arbitrate(buyer_refund, ruling_hash)` com qualquer split.

4.3 Freelancer

Arquivo: `soroban-contracts/freelancer/src/lib.rs`

Diferencial: milestone com valor próprio

```
pub milestone_amounts: Vec<i128>, // [3000, 4000, 5000, 3000]
```

Cada entrega pode ter um valor diferente. Antes era `total / count` .

Renegociação com dupla assinatura

```
pub fn propose_change(env: Env, proposer: Address, new_total: i128, new_count: u32) {
    // ... salva proposta no storage
}

pub fn accept_change(env: Env, acceptor: Address) {
    // ... acceptor != proposed_by
    // ... cobra/devolve delta automaticamente
}
```

Proteção contra abandono

```
pub fn withdraw_unspent(env: Env) {
    let now = env.ledger().timestamp();
    if now < p.last_activity_ts + p.stale_secs {
        panic_with_error_freel(&env, FreelError::NoStaleness);
    }
    // ... cliente recupera o saldo
}
```

`last_activity_ts` é atualizado a cada `submit_delivery` , `approve_delivery` e `reject_delivery` . Se o freelancer some por 30 dias, cliente recupera.

4.4 Legal Fees (Honorários Advocatícios)

Arquivo: `soroban-contracts/legal-fees/src/lib.rs`

Três componentes de pagamento

1. `pay_retainer()` — entrada (one-shot)

2. `pay_monthly()` — mensal por `duration_months`
3. `confirm_success()` — % sobre valor recuperado (quota litis)

Por que dupla assinatura no êxito?

A norma da OAB exige que o cliente confirme o valor recuperado, evitando que o advogado "infle" o ganho para receber mais. Implementação:

```
pub fn propose_success(env: Env, proposer: Address, recovered_amount: i128) {
    // ... salva SuccessProposal no storage
}

pub fn confirm_success(env: Env, confirmer: Address) {
    if confirmer == prop.proposed_by {
        panic_with_error(&env, CommonError::Unauthorized);
    }
    // ... transfere success_rate_bps% do recovered_amount
}
```

Multa de distrato

```
let remaining_months = (d.duration_months - d.months_paid) as i128;
let remaining_value = d.monthly_fee.saturating_mul(remaining_months);
let penalty = remaining_value * d.termination_fee_bps as i128 / BPS_DENOM;
```

Quem terminar paga multa para a contraparte. Default 10%.

4.5 Construction (Empreitada)

Arquivo: `soroban-contracts/construction/src/lib.rs`

Tri-assinatura por marco

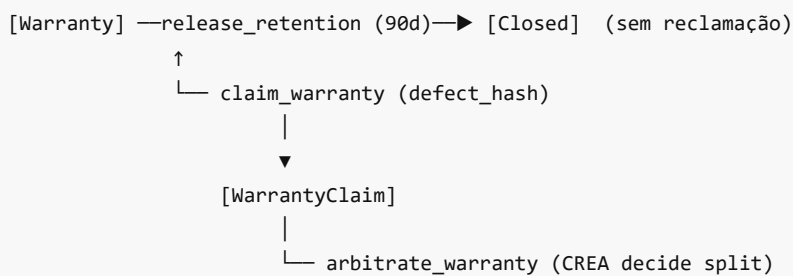
```
Construtora → submit_milestone(idx, laudo_hash)
↓
Engenheiro → engineer_sign(idx)
↓
Cliente → client_release(idx) → libera (1 - retention_bps)% do marco
```

Retenção de garantia

Padrão na construção civil. 5-10% do valor de cada marco fica retido até o aceite + warranty (90 dias).

```
let retain = m.amount * p.retention_bps / BPS_DENOM;
let pay = m.amount - retain;
token_transfer(env, asset, contract_addr, contractor, pay);
p.retention_locked += retain;
```

Warranty (garantia)



4.6 Real Estate Token (Tokenização Imobiliária)

Dois contratos trabalhando juntos:

Contrato	Papel
real-estate-share	Token SEP-41 — representa as cotas
real-estate-vault	Vault — captação, distribuição, venda

Por que separar?

O share token precisa ser SEP-41 padrão para ser **fungível** — investidores podem transferir cotas entre si, listar em DEX, etc. O vault é a lógica de negócio (não-fungível, single-instance).

O problema do Soroban: não dá pra iterar storage

Se temos 1000 cotistas e o sponsor distribui R\$ 50.000 de aluguel, **não dá** pra fazer um `for holder in holders { pay(holder, share) }` on-chain. Cada acesso a storage custa fee.

Solução: Merkle proofs.

Sponsor (off-chain):

1. Indexer faz snapshot: { alice: 30 shares, bob: 20 shares, carol: 50 shares }
2. Calcula alocações: { alice: 1500, bob: 1000, carol: 2500 }
3. Constrói Merkle Tree com folhas `keccak(period || holder || amount)`
4. Pega o root (32 bytes)

Sponsor (on-chain):

```
vault.distribute_rent(total=5000, merkle_root=0xab...)
```

Cada holder (on-chain):

```
vault.claim_rent(holder=alice, period=1, amount=1500, proof=[...])
```

↓

Contrato verifica:

```
folha_alice = keccak(1 || alice || 1500)
computed = walk(folha_alice, proof)
require computed == merkle_root
```

Vantagem: independente de quantos cotistas, custo on-chain é constante.

Votação ponderada

```
pub fn vote_sale(env: Env, voter: Address, approve: bool) {
  // ... cross-contract call para ler balance no share_token
  let balance: i128 = env.invoke_contract(
    &v.share_token,
    &Symbol::new(&env, "balance"),
    (voter,).into_val(&env),
  );
  if approve {
    prop.votes_yes_shares += balance as u32;
  } else {
    prop.votes_no_shares += balance as u32;
  }
}
```

Quorum + maioria simples decidem.

5. Como funciona o deploy ponta a ponta

Passo 1 — Você compila o WASM

```
cd soroban-contracts
./scripts/build-all.sh
```

Saída em `dist/rent.wasm` , `dist/ecommerce.wasm` , etc.

Passo 2 — Você sobe os WASMs para o Supabase Storage

```
./scripts/upload-wasms.sh
```

O bucket fica assim:

```
contracts-wasm/
  rent.wasm
  ecommerce.wasm
  freelancer.wasm
  legal_fees.wasm
  construction.wasm
  real_estate_vault.wasm
```

Passo 3 — Usuário cria contrato no frontend

No ContractEase, escolhe template "Aluguel", preenche o formulário, clica em "Implantar".

Passo 4 — Frontend chama a Edge Function

```
await deployContract({
  contractId: 'uuid-doc',
  templateId: 'rent',
```

```

    initArgs: [sc.struct({ landlord: sc.addr('G...'), ... })],
    network: 'testnet',
  });

```

Passo 5 — Edge Function executa 3 transações Stellar

```

// 1. Upload do WASM (sponsor paga ~0.5 XLM)
const uploadTx = TransactionBuilder()
  .addOperation(Operation.uploadContractWasm({ wasm }))
  .build();
// → retorna wasm_hash

// 2. Cria instância
const createTx = TransactionBuilder()
  .addOperation(Operation.createCustomContract({
    address: sponsor.publicKey(),
    wasmHash,
    salt,
  }))
  .build();
// → retorna contract_address (C...)

// 3. Invoca init com os args
const initTx = TransactionBuilder()
  .addOperation(contract.call('init', ...scArgs))
  .build();

```

Passo 6 — DB atualizado

```

UPDATE contracts SET
  soroban_contract_address = 'CXXXX...',
  soroban_wasm_hash = '0xab...',
  soroban_deploy_tx = '...',
  soroban_init_tx = '...',
  soroban_network = 'testnet',
  soroban_deployed_at = NOW()
WHERE id = 'uuid-doc';

```

Passo 7 — Usuários invocam ações

```

// Inquilino paga caução assinando com Freightier
await invokeAction({
  contractAddress: 'CXXXX...',
  method: 'pay_deposit',
  args: [],
  caller: tenantAddress,
});

```

A Freighter abre o popup, inquilino aprova, transação é enviada.

6. Frontend: chamando os contratos

6.1 Tipos de chamada

```
import { deployContract, invokeAction, readState, sc } from '@services/sorobanDeploy';
```

Função	Custo	Wallet	Uso típico
deployContract	sponsor paga	sponsor (Edge Function)	Cria nova instância
invokeAction	sponsor + caller	Freighter	Pagar, aprovar, etc.
readState	gratuito	nenhuma	Mostrar estado na UI
fetchEvents	gratuito	nenhuma	Timeline de transações

6.2 Helper `sc` para construir argumentos

```
sc.addr('GXXXX...') // → Address
sc.i128('25000000000') // → i128 (em stroops)
sc.u32(5) // → u32
sc.u64('17000000000') // → u64
sc.bool(true) // → bool
sc.str('texto') // → String
sc.bytes('ab12...') // → Bytes (hex sem 0x)
sc.opt(null) // → Option<T>::None
sc.opt(sc.addr('GXXXX')) // → Option<T>::Some
sc.vec([sc.i128('100'), sc.i128('200')]) // → Vec<T>
sc.struct({ field1: sc.u32(5), ... }) // → struct
```

6.3 Exemplo: monitorar estado de um aluguel

```
import { readState } from '@services/sorobanDeploy';

interface RentalAgreement {
  state: { tag: string };
  months_paid: number;
  monthly_rent: bigint;
  // ...
}

const agreement = await readState<RentalAgreement>(
  contractAddress,
  'get_agreement',
);

console.log('Mês atual:', agreement.months_paid);
```

```
console.log('Estado:', agreement.state.tag);
// → "Active", "Overdue", "Evaluation"...
```

6.4 Exemplo: inquilino paga aluguel

```
import { invokeAction } from '@services/sorobanDeploy';

try {
  const { txHash } = await invokeAction({
    contractAddress: 'CXXX...',
    method: 'pay_rent',
    args: [],
    // caller omitido → usa Freighter atual
  });
  console.log('Pagamento ok:', txHash);
} catch (err) {
  // Erros tipados do contrato chegam aqui
  if (err.message.includes('#102')) {
    alert('Aluguel já não está em atraso');
  }
}
```

7. Testando localmente

7.1 Pré-requisitos

```
rustup install stable
rustup target add wasm32-unknown-unknown
```

No Windows: você precisa do **Visual Studio Build Tools** com a workload "Desktop development with C++". Sem isso, o linker MSVC não funciona e `cargo build` falha mesmo para WASM (proc-macros precisam compilar para host).

7.2 Rodar os testes

```
cd soroban-contracts

# Todos os contratos
cargo test --workspace

# Um contrato específico, com output
cargo test -p contractease-rent -- --nocapture
```

7.3 Anatomia de um teste

```
#[test]
fn happy_path_full_cycle() {
```

```

let monthly_rent = 2500 * STROOP;
let fx = setup(monthly_rent, 3);

// 1. Estado inicial
assert_eq!(fx.contract.get_state(), RentState::AwaitingDeposit);

// 2. Inquilino deposita caução
fx.contract.pay_deposit();
assert_eq!(fx.contract.get_state(), RentState::Active);

// 3. 12 meses em dia
for _ in 0..12 {
    advance_time(&fx.env, 30 * 86_400);
    fx.contract.pay_rent();
}
// ...
}

```

Pontos a observar:

- `setup()` cria um ambiente isolado: token mock, partes, contrato.
- `env.mock_all_auths()` faz qualquer `require_auth()` passar.
- `advance_time()` manipula `ledger.timestamp` para testar timeouts.
- `assert_eq!` compara estado contra o esperado.
- Para testar erros: `let result = fx.contract.try_X(); assert!(result.is_err());`

7.4 Asset mock para os testes

```

let asset = env.register_stellar_asset_contract(admin);
let asset_admin = StellarAssetClient::new(&env, &asset);
asset_admin.mint(&tenant, &(monthly_rent * 36));

```

O Soroban SDK fornece uma implementação de SAC (Stellar Asset Contract) pronta para testes — comporta-se igual ao BRZ ou USDC real.

8. Como criar um novo contrato

Suponha que queremos adicionar `dental_treatment` (Tratamento Odontológico).

8.1 Crie a pasta no workspace

```

cd soroban-contracts
mkdir -p dental-treatment/src

```

8.2 Adicione no workspace `Cargo.toml`

```

members = [
    "common",
    "rent",

```

```
# ...  
  "dental-treatment", ← nova linha  
]
```

8.3 Crie o `dental-treatment/Cargo.toml`

```
[package]  
name = "contractease-dental-treatment"  
version.workspace = true  
edition.workspace = true  
# ... (copie de outro contrato)  
  
[lib]  
crate-type = ["cdylib", "rlib"]  
  
[dependencies]  
soroban-sdk = { workspace = true }  
contractease-common = { path = "../common" }  
  
[dev-dependencies]  
soroban-sdk = { workspace = true, features = ["testutils"] }
```

8.4 Esqueleto do `src/lib.rs`

Use este template:

```
#![no_std]  
  
use contractease_common::{panic_with_error, require_state, CommonError};  
use soroban_sdk::{contract, contracterror, contractimpl, contracttype, symbol_short,  
Address, Env, Symbol};  
  
const DATA: Symbol = symbol_short!("DATA");  
  
#[contracterror]  
#[derive(Copy, Clone, Debug, PartialEq, Eq, PartialOrd, Ord)]  
#[repr(u32)]  
pub enum DentalError {  
    // ... seus erros começam em 100  
}  
  
#[contracttype]  
#[derive(Clone, Debug, PartialEq, Eq)]  
pub enum DentalState {  
    Created,  
    InTreatment,  
    Completed,  
}  
  
#[contracttype]
```

```

#[derive(Clone)]
pub struct DentalContract {
    pub dentist: Address,
    pub patient: Address,
    // ... campos
    pub state: DentalState,
}

#[contracttype]
pub struct DentalInitParams {
    // ... mesmo shape do DentalContract sem `state`
}

#[contract]
pub struct DentalTreatment;

#[contractimpl]
impl DentalTreatment {
    pub fn init(env: Env, params: DentalInitParams) {
        // ... mesmo padrão do rent
    }

    // ... suas funções
}

#[cfg(test)]
mod test;

```

8.5 Adicione no scripts/build-all.sh

```

CONTRACTS=(
    "rent:contractease_rent"
    # ...
    "dental-treatment:contractease_dental_treatment" ← adicione
)

```

8.6 Adicione no mapeamento da Edge Function

apps/contractease/supabase/functions/deploy-soroban/index.ts :

```

const TEMPLATE_WASM: Record<string, string> = {
    // ...
    dental_treatment: 'dental_treatment.wasm',
};

```

8.7 Adicione o builder de init args

apps/contractease/src/services/sorobanInitArgs.ts :


```

export const SOROBAN_SUPPORTED_TEMPLATES = [
  // ...
  'dental_treatment',
] as const;

function buildDentalInitArgs({ vars, network }: BuildArgs): ScValPayload[] {
  return [
    sc.struct({
      dentist: sc.addr(vars.dentist),
      patient: sc.addr(vars.patient),
      // ...
    }),
  ];
}

// no switch:
case 'dental_treatment':
  return buildDentalInitArgs(args);

```

8.8 Compile e deploy

```

./scripts/build-all.sh
./scripts/upload-wasms.sh

```

Pronto. Próximo deploy de tratamento odontológico já vai virar contrato Soroban real.

9. Glossário

Termo	Definição
WASM	WebAssembly. Formato binário portátil onde Rust compila.
Stroop	Unidade mínima da Stellar (10^{-7} de unidade). Tudo é i128 em stroops on-chain.
SEP-41	Stellar standard para tokens fungíveis. Define transfer, balance, approve.
Soroban RPC	Endpoint que o frontend usa para enviar/simular transações.
Horizon	API REST tradicional da Stellar (não-Soroban). Usamos para anchor-on-stellar.
Memo	Campo de até 32 bytes em transações Stellar clássicas. Usado para ancoragem por hash.
bps (basis points)	1 bps = 0.01%. 2000 bps = 20%. Padrão em finanças para evitar floats.
Sponsor	Account que paga as fees de transação. Em Soroban, qualquer um pode patrocinar.
Freighter	Extensão Chrome de wallet Stellar. Equivalente do Metamask.
Storage instance	Storage do contrato com TTL renovado automaticamente.
Storage persistent	Storage indexado (mapas, listas). TTL precisa ser estendido manualmente.

Storage temporary	Storage barato para dados de curta duração (allowances).
Merkle proof	Prova de inclusão em árvore Merkle. Usada no vault imobiliário.
Quota litis	Honorário por % do êxito (norma OAB).
ART/RRT	Anotação de Responsabilidade Técnica (CREA/CAU).

10. Troubleshooting

"linker link.exe failed" no Windows

Você não tem Visual Studio Build Tools instalado. Solução:

1. Baixe em <https://visualstudio.microsoft.com/visual-cpp-build-tools/>
2. Instale a workload "Desktop development with C++"
3. Reabra o terminal e rode `cargo build` de novo.

"STELLAR_SECRET_KEY não configurada"

A Edge Function não tem o secret. Configure:

```
supabase secrets set STELLAR_SECRET_KEY=SXXXXXX...
```

A chave começa com `S`. Pegue com `stellar keys show sponsor --secret`.

"Account not found" no deploy

A conta sponsor precisa existir na testnet. Funde com friendbot:

```
https://friendbot.stellar.org/?addr=G<publickey>
```

Erro #1 (Unauthorized)

Você chamou uma função sem assinar com a wallet correta. Verifique:

- A função tem `require_auth()` em alguma `Address`.
- A Freighter está conectada com a wallet certa.
- A wallet é a esperada pelo contrato (ex: `tenant` em `pay_deposit`).

Erro #8 (AlreadyInitialized)

Você tentou chamar `init` duas vezes. Cada contrato só inicializa uma vez. Para "reiniciar", precisa fazer deploy de uma nova instância.

Contract address mudou após o deploy?

Não. O contract address é determinado pelo `salt` no `createContract`. Se você guardou `CXXX...` no banco, é esse para sempre.

"Simulation failed" no readState

A função read-only não existe no contrato OU os argumentos estão errados. Verifique o nome e os tipos.

Conclusão

Você agora tem:

- **6 smart contracts profissionais** em Rust com testes (~3.500 linhas).
- **Infra de deploy** completa (Edge Function + frontend service).
- **Mapeamento de variáveis** template → init args.
- **Documentação** suficiente para evoluir sozinho.

O próximo passo é apenas operacional: instalar Visual Studio Build Tools, buildar os WASMs, configurar o sponsor e fazer o primeiro deploy real.

Em caso de dúvida na hora de evoluir o código:

1. Comece lendo o `lib.rs` do contrato mais parecido com o que você quer.
2. Copie o esqueleto e adapte.
3. Escreva o teste primeiro (TDD funciona muito bem em Soroban).
4. Build local — `cargo test -p contractease-X` — antes de qualquer push.

Boa construção! 🚀